



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



Code of Conduct:

I Rakcha A(192311421) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Rakcha A



SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA3711

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Q1. A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.

Answer:

Testing as an Engineering Activity

Testing is not a last-minute activity of 'trying the software'; it is an engineering discipline with planning, design, execution, measurement and improvement. Applied to the banking application, this means replacing random checking with a defined, repeatable and measurable test process.

Redesigned Approach

- Test planning: define scope, objectives, risks, entry/exit criteria, schedule and resources for modules such as login, fund transfer, balance enquiry and statements.
- Systematic test design: derive test cases from requirements using techniques such as equivalence partitioning, boundary value analysis, decision tables and state transition testing; maintain a requirements-to-test traceability matrix.
- Tools: a test management tool (Jira/TestRail) for cases and defects, Selenium/Postman for automated functional and API regression, JMeter for load, and a CI server (Jenkins) to run suites on every build.
- Standards: follow IEEE 829 for test documentation, IEEE 1044 for defect classification, ISO/IEC 25010 for quality attributes, and organisational templates and review checklists.



How It Improves Defect Detection and Quality

Test cases derived from requirements give measurable coverage, so no feature is silently skipped. Reviews of test cases catch gaps early, automation makes regression repeatable after every change, and metrics (defect density, defect detection percentage) show where quality is weak. Defects are therefore found earlier and cheaper.

Techniques to Replace Ad-hoc Testing

Black-box techniques (equivalence partitioning, boundary value analysis, decision tables, state transition), white-box techniques (statement and branch coverage), risk-based testing for critical transactions, and experience-based but structured approaches such as session-based exploratory testing with charters.

Practical Example

Consider a transfer rule: amount must be between Rs. 1 and Rs. 1,00,000 per transaction. Ad-hoc testers might try only Rs. 5,000. Using boundary value analysis, the engineer tests 0, 1, 2, 99,999, 1,00,000 and 1,00,001, plus invalid inputs such as negative numbers and letters. A defect where Rs. 1,00,000 is wrongly rejected (a '<' used instead of '<=') is caught immediately, whereas the ad-hoc approach would miss it until a customer complained. Conclusion: engineering-style testing yields higher coverage, fewer production failures and traceable evidence of quality.

Q2. An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.

Answer:

Structured Testing Across SDLC Phases

Testing activities are attached to every SDLC phase (V-model style) so that quality is built in rather than inspected at the end.

- Requirements phase: review requirements for completeness and testability; prepare the acceptance test plan. Relevance: ambiguous requirements are the root of many defects.
- Design phase: review architecture and design; prepare system and integration test plans. Relevance: design flaws are removed before coding.



- Coding phase: code reviews, static analysis and unit testing by developers. Relevance: the cheapest place to find logic errors.
- Integration phase: test interfaces between card reader, PIN module, bank server and cash dispenser.
- System testing: end-to-end functional and non-functional tests (performance, security, recovery).
- Acceptance testing: validate against business needs with users/bank staff.
- Maintenance: regression testing after every fix or enhancement.

Application to ATM Withdrawal

- Requirements: withdrawal only after valid card and PIN; amount in multiples of 100; amount cannot exceed balance or daily limit; receipt and balance update after dispensing.
- Unit: validate amount-multiple logic, limit calculation and balance deduction functions.
- Integration: confirm that debit at the bank server and dispensing by the hardware happen together; test timeout between modules.
- System: test cases such as valid withdrawal, insufficient balance, amount not multiple of 100, three wrong PINs, cash out in machine, network failure mid-transaction (balance must not be debited without cash dispensed).
- Acceptance and regression: a bank officer verifies scenarios; the automated regression suite is re-run on each release.

How Process Consistency Reduces Defects

A defined process means every build goes through the same reviews, test levels, entry/exit criteria and defect reporting. Results become repeatable, so inconsistent behaviour is traced to a cause instead of being rediscovered by chance. Early detection lowers rework cost, and metrics allow continuous improvement.

Expected Outcomes

Fewer inconsistent withdrawals and reconciliation errors, higher test coverage, early defect detection, reduced production incidents, better traceability and increased customer trust.

Q3. A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how



improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.

Answer:

Basic Definitions

- Error (mistake): a human action that produces an incorrect result, e.g., a developer misreads the requirement that a password is case-sensitive.
- Defect (fault, bug): the flaw in the code or document caused by the error, e.g., the login code converts the password to lower case before comparison.
- Failure: the observable deviation of the system when the defect is executed, e.g., a user entering 'Pass@123' is rejected, or accepted with 'PASS@123'.

Relationship

Error leads to defect, and defect leads to failure (only when the faulty code is executed under the right conditions). A defect may exist without ever causing a failure, and one defect may cause many failures.

Application to the Scenario

- Login: the expected result is access only for the correct credentials; actual result is access with the wrong case. The mismatch is the failure; the lower-casing statement is the defect; the developer's misunderstanding of the requirement is the error.
- Transaction: a user transfers Rs. 500 and the balance is reduced by Rs. 50. The failure is the wrong balance; the defect is a decimal/rounding or formula mistake in the code; the error is the developer's wrong handling of currency units.

Effect of Improper Understanding

If testers and developers use the terms loosely, a report such as 'login is wrong' does not say whether it is a requirement problem, a code problem or an environment issue. This causes wrong prioritisation, back-and-forth communication, defects marked 'cannot reproduce', and root causes that are never fixed because only the symptom is addressed.

Improving Defect Reporting Clarity

- Use a standard template: ID, title, module, environment/build, steps to reproduce, test data, expected result, actual result, severity, priority, screenshots/logs, status.
- Describe the failure (observed behaviour) objectively and, where known, the suspected defect location.
- Report one defect per report and keep titles specific.



Example of a clear report: 'Login accepts password in a different case. Steps: 1) Register with Pass@123, 2) log in with PASS@123. Expected: Invalid credentials. Actual: Logged in. Severity: High (security).' This allows the developer to reproduce and fix the issue quickly.

Q4. A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.

Answer:

Relevant Testing Principles

- Defect clustering: a small number of modules usually contain most defects (Pareto principle, roughly 80/20).
- Testing shows presence of defects, not their absence: passing tests do not prove correctness.
- Exhaustive testing is impossible: effort must be focused using risk and priority.
- Early testing: starting early reduces cost and clustering later.
- Pesticide paradox: repeating the same tests stops finding new defects, so tests must be reviewed and refreshed.
- Testing is context dependent: a payment module needs a different depth of testing than a help page.
- Absence-of-errors fallacy: a defect-free system that does not meet user needs is still a failure.

How They Guide Test Planning

Using defect data from earlier releases and the current cycle, the test manager identifies the high-defect modules and allocates more testers, more time and deeper techniques there. Risk analysis combines likelihood of failure (complexity, change frequency, past defects) with impact (money, safety, customers). The plan reserves time for retesting and regression of the hot modules and revises test cases regularly to avoid the pesticide paradox.

Prioritising Test Cases

- Rank by risk = probability x impact; run high-risk and business-critical cases first.
- Give priority to modules with the highest defect density and recent code churn.



- Prefer cases covering frequently used features and those linked to high-priority requirements.
- Use techniques such as risk-based testing, requirement coverage and regression test selection.

Efficiency Gains

Effort goes where defects actually are, so more defects are found per hour, critical defects are found early, schedule slippage reduces and effort is not wasted on low-risk areas.

Real-time Example

In an online shopping project, defect logs show that 70% of defects come from Payment and Cart modules. The team runs extra boundary, concurrency and failure-injection tests there, places those cases at the top of the execution order, reviews and updates them each cycle, and runs only a smoke suite on low-risk modules like 'About Us'. Defect leakage to production drops and release delays are reduced.

Q5. A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.

Answer:

Origin of Defects

Defects originate from human errors made at any lifecycle stage: unclear or missing requirements, faulty design, coding mistakes, incorrect integration, inadequate test environments, poor communication, time pressure and changes made without impact analysis.

Defect Classification

- Requirements/specification defects: missing, ambiguous or conflicting requirements.
- Design defects: wrong algorithm, interface or data-structure design.
- Coding defects: logic errors, control-flow errors, data-handling errors, boundary errors, syntax and initialisation faults.
- Testing defects: incorrect test cases or environment set-up.



- Also classify by severity (critical to low), priority, module, phase of origin and phase of detection, as in IEEE 1044.

Using the Repository for Test Design

Analysing the repository reveals patterns, for example, 30% of defects are boundary errors in input validation, or most defects come from the reports module. Testers then add boundary value cases and checklists for these types, focus regression on the high-defect modules, and convert every escaped defect into a permanent regression test case.

Using the Repository for Prevention

- Perform root cause analysis (e.g., 5-Whys, fishbone) on frequent defect classes.
- Update coding standards, review checklists and requirements templates based on findings.
- Share defect trends with developers in retrospectives.

Improvements to Avoid Recurrence

- Mandate complete, classified entries (origin, type, severity, root cause, fix, verification).
- Link defects to requirements, test cases and builds.
- Track metrics: defect density, defect removal efficiency, reopen rate, repeat-defect rate.
- Review the repository before test planning for each release.

Expected Quality Improvements

Reduced repeat defects, higher defect removal efficiency, better targeted test cases, shorter fix time, fewer production escapes and a measurable trend of improving quality across releases.

Q6. A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.

Answer:

Identifying the Root Cause

Applying the principle that testing must be risk-based and that defects cluster, the fund transfer module is analysed first. Steps: reproduce the crash with exact steps, collect logs and crash dumps, note device, OS version, network and data used, and narrow down the conditions.

- Check inputs: very large amounts, special characters, empty beneficiary, zero or negative values.



- Check state and resources: low memory, interrupted network, switching apps mid-transaction, session timeout.
- Check integration: delayed or malformed responses from the bank server or payment gateway, null pointer on an unexpected response.
- Apply root cause analysis (5-Whys): the app crashes, because a null response is not handled, because no error handling was coded, because the requirement did not state failure behaviour.

How Systematic Testing Reduces Such Issues

Designed test cases cover valid, invalid and boundary inputs, negative scenarios and non-functional conditions. Early unit and integration testing catches unhandled exceptions, regression testing ensures fixes do not break other features, and measured coverage reveals the untested paths where crashes hide.

Examples

- Boundary: transfer limit of Rs. 2,00,000; test 1, 2,00,000 and 2,00,001.
- Interruption: incoming call or network loss at the confirmation screen; the app must show an error and the balance must stay consistent.
- Double-tap on 'Pay' must not create two transactions; the app must not crash.
- Device matrix: low-end Android, old iOS, different screen sizes.

Improvements in Testing Practice

- Add negative, stress and fault-injection tests and a device/OS compatibility matrix (real devices and cloud farms).
- Automate regression with Appium/Espresso and run it in CI.
- Use crash analytics (Firebase Crashlytics) and feed production crashes into new test cases.
- Review error-handling in code reviews and include performance and security testing.

Q7. A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes.



Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.

Answer:

Testing versus Debugging

- Testing: the process of executing software (or reviewing artefacts) to detect failures and evaluate quality. Its goal is to find and report defects. It is performed mainly by testers (and developers at unit level), can be planned and automated, and is done with defined test cases.
- Debugging: the process of locating the cause of a known failure, correcting the defect and verifying the fix. Its goal is to remove the defect. It is performed by developers, is typically unplanned and driven by an observed failure.

In short, testing shows that a problem exists; debugging finds why and fixes it.

Roles in Software Quality

Testing provides evidence about quality and risk and drives decisions on release. Debugging removes defects and restores correct behaviour. Both are needed: without testing, defects remain unknown; without debugging, known defects remain unfixed.

Effect of Improper Usage

- If developers only 'debug' while coding, they test just the paths they expect to work and miss independent verification, so many defects escape.
- If testers try to find the cause and patch code, their time for test design is lost and unreviewed fixes introduce new defects.
- Fixes without retesting and regression leave defects reopened or cause side effects.

Structured Integrated Approach

1. Testing detects a failure and logs a complete defect report.
2. Developer reproduces the failure and debugs: locate the fault, analyse the cause, correct it.
3. Developer performs unit testing of the fix and updates the defect status.
4. Tester performs confirmation (re)testing of the failed case.
5. Tester executes regression tests to ensure no side effects.
6. Root cause data is recorded in the repository and used to improve tests and coding practices.



This clear hand-off, with defined roles and status, makes defect resolution faster and more reliable.

Q8. An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.

Answer:

Origin of Cross-browser Defects

- Different rendering engines (Blink, WebKit, Gecko) interpret HTML/CSS differently.
- Use of non-standard or unsupported JavaScript APIs, CSS properties or HTML5 features.
- Differences in default styles, fonts, cookie and storage handling, and security settings.
- Browser versions, operating systems, screen sizes and plug-in or extension interference.
- Developers testing only on one preferred browser, so errors originate from incomplete requirements and compatibility assumptions.

Strategy

- Risk-based selection: use analytics to choose the browsers, versions, OS and devices used by most customers (e.g., Chrome, Edge, Safari, Firefox; desktop and mobile).
- Define a compatibility matrix and prioritise critical flows: search, product page, cart, checkout, payment, login.
- Design test cases once and execute across the matrix; automate the stable ones for regression.
- Check functional behaviour, layout and responsiveness, performance and accessibility.
- Test early and with every build, and re-run when browser versions update.

How It Improves Reliability

Example: the 'Place Order' button works in Chrome but is hidden in Safari because of an unsupported CSS property. A matrix-driven test detects this before release, the defect is logged with browser details and fixed with a standard-compliant alternative. Systematic coverage means fewer customer-visible failures, fewer abandoned carts and consistent user experience.

Tools and Techniques

- Selenium WebDriver/Selenium Grid, Cypress and Playwright for automated cross-browser tests.



- BrowserStack, LambdaTest or Sauce Labs for real-device and browser cloud testing.
- Visual regression tools (Percy, Applifools), Lighthouse for performance and accessibility.
- Techniques: equivalence partitioning of browser/OS combinations, pairwise (combinatorial) testing to reduce the matrix, and progressive enhancement and graceful degradation checks.

Q9. A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.

Answer:

Categorising the Defects

Logical errors in salary calculation fall under coding defects (algorithmic and processing defects) in the standard defect classification:

- Algorithmic/logic defects: wrong formula, e.g., net salary = basic + HRA - tax computed with the wrong order of operations or wrong tax slab condition.
- Control-flow defects: wrong condition or loop, e.g., overtime paid when hours ≥ 40 instead of > 40 ; missing 'else' path for part-time employees.
- Data defects: wrong data type or rounding, e.g., integer division truncating decimals; incorrect initialisation of the deduction variable.
- Boundary defects: errors at slab limits such as income exactly at Rs. 5,00,000.
- Requirement-origin defects: if the tax rule itself was misunderstood. Also classify by severity (high: monetary and legal impact) and by phase of origin.

How Understanding Defect Classes Improves Testing

Knowing that logic and boundary defects dominate guides testers to choose the matching techniques: boundary value analysis for slabs, decision tables for rule combinations, and branch/condition coverage for control-flow errors. It also helps prioritise severe defect classes and design checklists for reviews.

Prevention in Future Releases

- Peer code reviews and static analysis with a checklist for formulas and conditions.
- Unit tests with expected values from independently calculated spreadsheets.



- Root cause analysis and updating coding guidelines; use of well-tested common functions for tax and rounding.
- Requirement walkthroughs with the HR/finance domain experts.
- Add every found defect as a regression test.

Improvements in Testing Strategy

- Decision table testing for combinations of grade, allowance, leave and tax rules.
- Boundary tests at each slab and parallel testing against the old system or a reference spreadsheet.
- Automated regression for monthly payroll runs, with test data covering mid-month joiners, leavers and arrears.
- Track repeat-defect rate and defect density per module in the repository.

Q10. A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

Answer:

Redesigning the Process as an Engineering Activity

The team should follow engineering steps: analyse requirements, plan, design test cases, set up the environment, execute, log defects, report and improve. Test cases are reviewed and baselined before execution, and results are measured.

Importance of Structured Test Case Design

- Ensures coverage of every requirement and traceability.
- Makes testing repeatable, so results can be compared across builds and testers.
- Reduces redundant tests and saves time; finds defects more efficiently.
- Supports reviews, automation, regression and knowledge transfer.
- Provides evidence of quality for stakeholders.

A test case includes ID, objective, preconditions, input data, steps, expected result, actual result and status.

Improvement in Defect Detection (Example)

For a registration form with age valid from 18 to 60, random testing may try ages 25 and 30 only. Structured design using equivalence classes (<18, 18-60, >60) and boundary values (17,



18, 19, 59, 60, 61) quickly reveals that age 60 is rejected because of a '<' in the code. A planned set of 8 cases finds the defect that dozens of random inputs missed.

Techniques to Enhance Effectiveness

- Black-box: equivalence partitioning, boundary value analysis, decision tables, state transition, use-case testing.
- White-box: statement, branch and condition coverage.
- Experience-based: error guessing and structured exploratory testing with charters.
- Risk-based prioritisation, peer review of test cases, a traceability matrix and periodic refresh of tests to avoid the pesticide paradox.

Q11. A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.

Answer:

Analysis Using Testing Principles

Because testing is context dependent and healthcare is safety-critical, the analysis is risk-based: patient data (demographics, prescriptions, lab results, billing) shared between Registration, Pharmacy, Lab and Billing modules is the highest-risk area. Defects are expected to cluster at module interfaces, and early testing of data flow reduces cost and risk.

Origin of Defects

- Requirements: unclear ownership of master data and missing data-sharing rules.
- Design: duplicated tables, missing foreign keys or constraints, inconsistent data formats (date or unit formats).
- Coding: field mapping errors, truncation, missing transaction commit/rollback, no validation.
- Integration: different modules updating the same record without synchronisation; interface version mismatches.
- Changes made in one module without impact analysis on others.

How Systematic Testing Ensures Data Integrity

- Integration testing of data flows end to end: a patient registered in Registration must appear identically in Pharmacy, Lab and Billing.



- CRUD tests: update an allergy in one module and verify it everywhere; delete or merge records and check referential integrity.
- Database validation with SQL queries, boundary and format tests (units such as mg versus g), concurrency tests and rollback tests on failure.
- Security and audit-trail tests; regression after every change; use of anonymised test data.

Example: a patient's blood group updated as 'O+' in Registration appears as 'O' in Lab. Integration testing detects the truncation defect before release.

Improvements in Defect Tracking and Reporting

- Use a tracking tool with fields for module, data entity affected, severity (patient-safety impact), steps, expected/actual data, and root cause.
- Link defects to requirements and test cases; set severity-based SLAs.
- Hold regular defect triage meetings and review trends and data-related defect classes from the repository.

Q12. A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.

Answer:

Analysing the Issue with a Defect Repository

A defect repository is a central, structured store of all defects with their attributes and history. Repeated defects suggest that the repository is not being analysed, root causes are not removed, and fixes are not converted into regression tests. Analysis steps: extract defects of the last releases, group by module, type, origin phase and root cause, and identify the defects that recurred or were reopened.

Support for Test Design

- Defect-type statistics guide technique selection (e.g., many boundary defects lead to more boundary value tests).
- High-defect modules receive more test cases and deeper coverage.
- Each escaped defect becomes a new test case in the regression suite.
- Past defects are used to build checklists and to support error guessing.



Identifying Defect Patterns

Use Pareto charts to show the 20% of modules or defect types causing 80% of defects, trend charts across releases, defect density per module, and cause-and-effect or 5-Whys analysis. Example: the repository shows that 40% of defects in four releases are 'missing input validation' in forms. This pattern shows a systematic gap rather than random mistakes.

Improvements to Prevent Recurrence

- Perform root cause analysis for every major defect and implement corrective and preventive actions.
- Update coding standards, review checklists, requirement templates and test design guidelines.
- Maintain complete, consistent defect data and link defects to tests, requirements and builds.
- Monitor metrics: repeat-defect rate, defect removal efficiency, reopen rate and escape rate.
- Review repository trends in each release planning and retrospective meeting, and train the team on recurring defect classes.

Q13. A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.

Answer:

Identifying Gaps

Using the principles that exhaustive testing is impossible, testing is context dependent and early testing saves cost, the gap analysis shows that the testing was functional and low-volume, with no realistic load, stress, concurrency or scalability scenarios. Non-functional requirements (e.g., 5,000 simultaneous users during exams) were probably not defined, and the test environment did not resemble production.

Impact of Test Planning and Coverage

A good test plan lists non-functional objectives, load profiles, tools, environments, risks and exit criteria. Coverage must include not only requirements and code but also load conditions, user behaviours and data volumes. Where coverage of peak scenarios is absent, bottlenecks



(database connections, memory leaks, session handling, server limits) remain hidden until production.

How Better Strategies Improve Reliability

- Performance testing: baseline response times under normal load.
- Load testing: expected peak (e.g., 5,000 students logging in at 9:00 for an exam).
- Stress and spike testing: beyond peak, to find the breaking point and verify graceful recovery.
- Endurance (soak) testing: long runs to find memory leaks.

Example: a load test shows that response time rises from 2 s to 30 s after 2,000 users because the database pool is limited to 100 connections. Increasing the pool and adding caching removes the failure before release.

Enhancements

- Define measurable performance requirements and use production-like environments.
- Use JMeter, Gatling or Locust, and monitoring tools (Grafana, New Relic) in test and production.
- Add performance tests to CI/CD, use auto-scaling and CDN, and plan capacity from past usage data.
- Run periodic regression and review of the test strategy after each incident.

Q14. A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

Answer:

Role of the Tester across SDLC

- Requirements: review requirements for clarity, completeness and testability; identify ambiguities and acceptance criteria.
- Design: review designs and identify risks; start test planning and test design.
- Coding: help in unit test and code review guidelines, prepare test data and environments.
- Testing phases: execute tests, report defects, verify fixes.
- Release and maintenance: acceptance support, regression testing, analysis of production defects.



Impact of Early Involvement

Following the principle of early testing, defects found in requirements or design cost much less to correct than those found in testing or production (the cost can increase ten times or more at each later phase). Early involvement also improves requirement quality, creates shared understanding, shortens test preparation time and reduces rework and delays.

Practical Example

In a ticket-booking project, the tester reviewing the requirement 'cancel ticket and refund as per policy' asks for the exact refund percentages and time slabs. The missing detail is added before coding. Without this review, developers would have assumed a rule, the defect would have been found at system testing, and design and code changes would have been required. Early involvement saves effort, avoids delays and improves quality.

Changes in Collaboration Practices

- Include testers in requirement reviews, sprint planning, backlog refinement and design discussions.
- Adopt Agile/DevOps practices: shift-left testing, three-amigos sessions (business, developer, tester), BDD with shared acceptance criteria.
- Hold daily stand-ups and joint defect triage; share tools and dashboards.
- Promote shared ownership of quality, cross-skilling, and continuous integration with automated tests running on every commit.

Q15. A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.

Answer:

Identifying Gaps

Since testing can show only the presence of defects and exhaustive testing is impossible, coverage must be selected carefully. To find gaps, compare test cases with requirements (requirement traceability matrix), measure code coverage (statement, branch, condition) with tools such as JaCoCo or Coverage.py, and review defects found in production to see which scenarios were never tested. Typical gaps are missing negative tests, boundaries, combinations of inputs and error paths.



How Proper Coverage Ensures Reliability

Coverage means each requirement, input class, decision outcome and important path is exercised at least once. This increases confidence that the system behaves correctly in all intended situations, not just the common ones, and reduces the chance that untested paths fail in production. It is a measure of testing thoroughness, though 100% coverage does not guarantee absence of defects.

Improving Test Case Design

Example: a discount rule says 'orders of Rs. 1,000 or more get 10% discount; members get an additional 5%'. Tests with only Rs. 1,500 and a member give correct results. Better design: equivalence classes (below 1,000, 1,000 and above), boundary values (999, 1,000, 1,001), and a decision table of member/non-member against the amount condition, giving 6 cases. Testing 1,000 may reveal a '>' versus '>=' defect that earlier tests missed.

Techniques for Better Validation

- Equivalence partitioning, boundary value analysis, decision tables, state transition and pairwise testing.
- Branch and condition coverage for decision logic; path testing for critical flows.
- Negative and error-handling tests; test case reviews and inspections.
- Mutation testing to check the strength of the tests, and exploratory testing for unexpected cases.
- Regular coverage reports and updating test cases to avoid the pesticide paradox.

Q16. A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.

Answer:

Improving Defect Reporting

The delay arises because reports are vague, incomplete or lack reproducible steps, so developers spend time asking questions or cannot reproduce the problem. Applying testing concepts (error, defect, failure) the tester reports the observed failure precisely, along with the evidence needed to locate the defect.



Role of Clear Definitions in Tracking

- A common vocabulary (error, defect, failure, severity, priority, status) avoids misunderstanding between testers, developers and managers.
- Defined lifecycle states (New, Assigned, Open, Fixed, Retest, Verified, Closed, Reopened, Rejected, Deferred) make progress visible.
- Severity (impact on the system) and priority (urgency of fixing) are defined separately so that scheduling is correct.
- Classification by type and module enables trend analysis and metrics.

Structured Reports Improve Communication

A structured report contains: ID, title, module, build/environment, steps to reproduce, test data, expected versus actual result, severity/priority, attachments (screenshots, logs), reporter and date.

Poor report: 'Tracking not working.' Structured report: 'Shipment tracking shows Delivered for an order still In Transit. Steps: 1) Login as customer, 2) open order 4521, 3) click Track. Expected: In Transit. Actual: Delivered. Build 2.3, Chrome 120. Severity: High, Priority: P1. Log attached.' The developer can reproduce and fix it immediately, reducing delay.

Improvements in Reporting Standards

- Adopt a standard template and tool (Jira/Bugzilla) in line with IEEE 829/1044.
- Make mandatory fields compulsory and give writing guidelines: one defect per report, objective language, minimal reproducible steps.
- Review report quality in triage meetings, set SLAs for response and fix times, and use dashboards for aging and status.
- Train testers and developers in the reporting process.

Q17. A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.

Answer:

Process-based Testing Approach

Process-based testing defines a standard, repeatable testing process that every module follows: planning and control, test analysis and design, implementation, execution, defect reporting,



evaluation of exit criteria and test closure. Models such as TMMi or ISO/IEC/IEEE 29119 provide the framework.

How Standardised Processes Improve Quality

- All modules receive the same level of rigour, which removes dependence on individual tester style.
- Results are comparable and measurable, enabling fair decisions and process improvement.
- Reviews and entry/exit criteria prevent poor quality builds from moving forward.
- New team members become productive faster, and rework and omission of tests reduce.

Example

In a university ERP, the Admissions team writes detailed test cases and tracks coverage, while the Fees team tests informally. After a standard process is adopted, both modules use the same test plan template, traceability matrix, test case format (ID, steps, expected result), defect lifecycle and exit criteria (100% critical cases executed, no open high-severity defects). Fee calculation defects, previously missed, are now found because cases are designed and reviewed against requirements.

Methods to Ensure Consistency

- Organisation-wide test policy, strategy and templates.
- Common test management and defect tools, and checklists for reviews.
- Process audits and peer reviews of test artefacts; quality gates before each phase.
- Common metrics (coverage, defect density, defect removal efficiency) reviewed in regular meetings.
- Training, a central test lead or test centre of excellence, and continuous improvement through retrospectives.

Q18. A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such



scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.

Answer:

Why Edge Cases Were Missed

- Exhaustive testing is impossible, and without technique-based design the team tested only typical values.
- Requirements did not specify limits, empty values or error behaviour, so test cases were not derived.
- Time pressure, lack of risk analysis and test data that did not represent real use.
- The pesticide paradox: the same tests were repeated, so new classes of problems were not found.

How Boundary Testing Helps

Defects are very common at the edges of input ranges, because of mistakes such as '<' instead of '<=' or off-by-one errors in loops and arrays. Boundary value analysis tests values at, just below and just above each boundary, so these defects are found with few test cases.

Including Edge Cases in Testing

Example: a field accepts a quantity from 1 to 100. Test values: 0, 1, 2, 99, 100, 101. Further edge cases: empty input, negative number, decimal, very large number, special characters, maximum field length, leap-year date (29 Feb), end-of-month date, zero balance, null or missing data, first and last records in a list, and simultaneous operations. If the system wrongly accepts 101 or rejects 100, the defect is detected before deployment.

Improvements in Test Design

- Use boundary value analysis together with equivalence partitioning, and error guessing from past production defects.
- Require limits and error behaviour in the requirements; include edge cases in the review checklist.
- Use data-driven automated tests, property-based or fuzz testing to generate extreme inputs.
- Perform risk-based prioritisation and refresh tests regularly; add each escaped defect as a regression test.



Q19. A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.

Answer:

Applying the Defect Repository Concept

The company should set up a central defect repository (Jira, Bugzilla, Azure DevOps) where every defect is recorded with a unique ID and a complete set of attributes: summary, description, steps to reproduce, environment/build, expected and actual results, severity, priority, type, module, origin phase, assignee, status, attachments, root cause and resolution. All defects pass through a defined lifecycle and are never tracked in emails or chats.

How Documentation Improves Defect Management

- Reproducibility: developers can reproduce and fix faster.
- Accountability and visibility: ownership, status and ageing are clear to managers.
- Avoids duplicate and lost defects, and supports prioritisation using severity and priority.
- Provides history for audits, release decisions and learning, and allows verification of fixes.

Repository-supported Analysis

Structured fields allow queries and reports: defects by module (find hot spots), by type or root cause (find process weaknesses), by severity and ageing, defect density, defect removal efficiency and trends over releases. Example: filtering shows that 35% of high-severity defects are in the 'Order Processing' module with the root cause 'incorrect validation', so the team strengthens validation tests and reviews there.

Improvements in Documentation Practices

- Standard defect template and mandatory fields, with guidelines for clear writing.
- Defined defect lifecycle and workflow, with triage meetings.
- Linking defects with requirements, test cases and code changes.
- Periodic audits of defect quality, training, dashboards and metrics, and mandatory root cause entry on closure.
- Following IEEE 1044 classification to keep data consistent.



Q20. A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.

Answer:

Redesigning the Testing Approach with Testing Principles

- Testing shows presence of defects: use planned, evidence-based testing and do not release on the basis of a few passing tests.
- Exhaustive testing is impossible: apply risk-based prioritisation; transaction processing (transfer, deposit, withdrawal, payment) is the highest-risk area.
- Early testing: review requirements and design for transaction rules and start test design early.
- Defect clustering: focus on modules with a history of failures, for example, settlement and reconciliation.
- Pesticide paradox: review and update tests every release.
- Context dependence: stress accuracy, security, atomicity and audit for banking.
- Absence-of-errors fallacy: validate against real business needs with acceptance testing.

How Structured Testing Improves Quality

A test plan, designed cases with traceability, test levels (unit, integration, system, acceptance), defined entry and exit criteria, formal defect management and metrics make results repeatable and measurable. Coverage becomes visible, defects are found earlier and fixed with regression protection.

Examples

- Atomicity: transfer Rs. 10,000 from A to B; if credit to B fails, debit from A must be rolled back. Test by injecting a failure after debit.
- Concurrency: two simultaneous withdrawals from the same account with a balance of Rs. 10,000 must not overdraw.
- Boundary: transaction limits, zero and negative amounts; duplicate submission by double-click; timeout and retry must not double-debit (idempotency).
- Data integrity: ledger totals before and after a batch of transactions must reconcile.



Techniques to Enhance Defect Detection

- Boundary value analysis, equivalence partitioning, decision tables and state transition testing (pending, success, failed, reversed).
- Integration and end-to-end tests with the payment gateway and core banking; API testing with Postman.
- Load and stress tests (JMeter) for month-end peaks; security testing for authorisation and tampering.
- Automated regression in CI, test case reviews, exploratory testing and root cause analysis of production incidents.

Result: fewer transaction failures, reduced financial risk and higher customer confidence.



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand